

Reviewer Pack — Minimal Frobenius Norm of Polynomial Representations and Cir...

Entry 7bf54c49f3e6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 15:27:54 UTC

Minimal Frobenius Norm of Polynomial Representations and Circuit Monotone Width

Entry ID: 7bf54c49f3e6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 15:27:54 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Frobenius Norm) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For any given Boolean circuit with n inputs, the Frobenius norm of its polynomial representation is bounded by $O(n^{3/2})$ and linearly correlated with its monotone width, such that $\|P(C)\|_F = \Theta(w_m(C))$, where $P(C)$ is the polynomial representation of the circuit C , $w_m(C)$ is the monotone width of C , and $\|\cdot\|_F$ denotes the Frobenius norm.

Rationale (proposer's reasoning):

The Frobenius norm has been used in algebraic combinatorics to study the complexity of representations of various combinatorial objects. By connecting it with circuit monotone width, we may uncover a new perspective on Boolean function complexity, potentially leading to improved lower bounds for monotone circuits.

Taxonomy category: Frobenius_Norm_to_Circuit_Monotone_Width (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5ddaaaa4d71a057c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

We consider a conjecture supported if, across at least 25 of 30 random seeds, the mean correlation coefficient between the Frobenius norm $\|P(C)\|_F$ and monotone width $w_m(C)$ for circuits with n inputs is ≥ 0.8 , with no single seed producing a correlation coefficient < 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Frobenius norm" AND "polynomial representation" AND "Boolean circuit complexity" - "monotone width" IN "Boolean circuits" AND "Frobenius norm" - "Algebraic combinatorics" AND "circuit monotone width" AND polynomial

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_circuit(n, depth):
    circuit = []
    for _ in range(depth):
        gate_type = random.choice(['AND', 'OR'])
        inputs = [random.randint(0, 1) for _ in range(n)]
        circuit.append((gate_type, inputs))
    return circuit

def compute_polynomial_representation(circuit):
    n = len(circuit[0][1])
    poly_rep = [[0] * (2 ** n) for _ in range(n + 1)]

    def evaluate(gate_type, inputs):
        if gate_type == 'AND':
            return all(inputs)
        elif gate_type == 'OR':
            return any(inputs)
        else:
            raise ValueError("Invalid gate type")

    for i in range(2 ** n):
        input_vals = [(i >> j) & 1 for j in range(n)]
        result = evaluate(circuit[0][0], circuit[0][1])
        poly_rep[0][i] = result

    for layer in range(1, len(circuit)):
        gate_type, inputs = circuit[layer]
        for i in range(2 ** n):
            input_vals = [(i >> j) & 1 for j in range(n)]
```

```

        result = evaluate(gate_type, input_vals)
        poly_rep[layer][i] = result

    return poly_rep

def compute_frobenius_norm(poly_rep):
    n = len(poly_rep[0])
    norm = 0
    for i in range(n):
        norm += sum(x * x for x in poly_rep[i]) ** Fraction(1, 2)
    return norm

def compute_monotone_width(circuit):
    n = len(circuit[0][1])
    width = 0

    def is_monotone(gate_type, inputs):
        if gate_type == 'AND':
            return all(inputs)
        elif gate_type == 'OR':
            return any(inputs)
        else:
            raise ValueError("Invalid gate type")

    for i in range(2 ** n):
        input_vals = [(i >> j) & 1 for j in range(n)]
        if is_monotone(circuit[0][0], circuit[0][1]):
            width += 1

    return width

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        circuit = generate_random_circuit(n, depth=3)
        poly_rep = compute_polynomial_representation(circuit)
        frobenius_norm = compute_frobenius_norm(poly_rep)
        monotone_width = compute_monotone_width(circuit)

        results.append({
            "n": n,
            "frobenius_norm": frobenius_norm,
            "monotone_width": monotone_width
        })

    correlation_sum = 0
    for i in range(len(n_values)):
        for j in range(i + 1, len(n_values)):
            n1, n2 = n_values[i], n_values[j]
            frobenius_norm1, frobenius_norm2 = results[i]["frobenius_norm"], results[j]["frobenius_norm"]
            monotone_width1, monotone_width2 = results[i]["monotone_width"], results[j]["monotone_width"]

```

```

        if frobenius_norm1 == 0 or frobenius_norm2 == 0:
            correlation_sum += 0
        else:
            correlation_sum += (frobenius_norm1 * monotone_width2 - frobenius_norm2 * monotone_width1)

    mean_correlation = correlation_sum / (len(n_values) * (len(n_values) - 1))
    conjecture_holds = mean_correlation >= 0.8
    counterexample = "" if conjecture_holds else "correlation_too_low"

    return {
        "metric_name": "mean_correlation",
        "metric_value": mean_correlation,
        "instances_tested": len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_too_low\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8006d320.py", line 127, in <module>
    trial_result = run_trial(seed)
    ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8006d320.py", line 87, in run_trial
    frobenius_norm = compute_frobenius_norm(poly_rep)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8006d320.py", line 56, in compute_frobenius_norm

```

```
norm += sum(x * x for x in poly_rep[i]) ** Fraction(1, 2)
~~~~~^^^
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its intended task of calculating the correlation coefficients. | next: Re-run the test with proper error handling to ensure that it can complete without crashing and produce the necessary data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15822
2	propose	ollama_remote	glm4:latest	0	0	13283
3	preregistration	ollama_remote	glm4:latest	0	0	9292
4	novelty	ollama_remote	glm4:latest	0	0	11720
5	novelty	ollama_remote	glm4:latest	0	0	15421
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15806
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20545
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14566
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16097
10	judge	ollama_remote	glm4:latest	0	0	16003

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 148557 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/7bf54c49f3e6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7bf54c49f3e6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7bf54c49f3e6.tar.gz` (if generated)